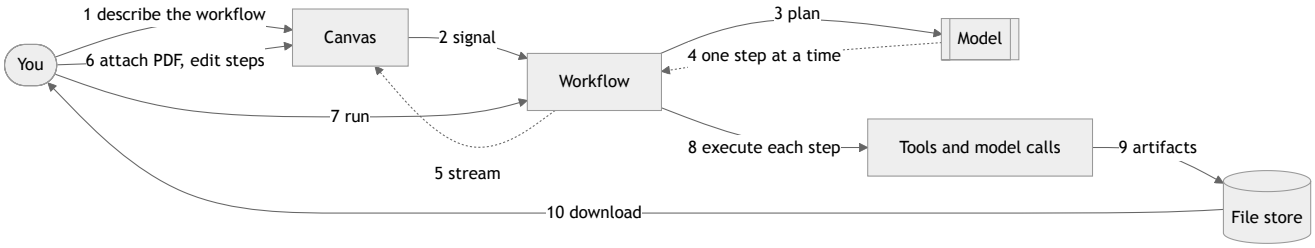
How it works, end to end

A lawyer has many recurring document jobs, and each is its own sequence. Reviewing an NDA is not the same as summarising an MSA, which is not the same as pulling every change of control clause out of a folder of agreements.

You describe the sequence you want, the model builds it on a canvas, you edit it and keep it. Then you run documents through it whenever that job comes up again.

Every claim in the output points back to a page in the source, and every one of those pointers has been checked by machine rather than trusted.

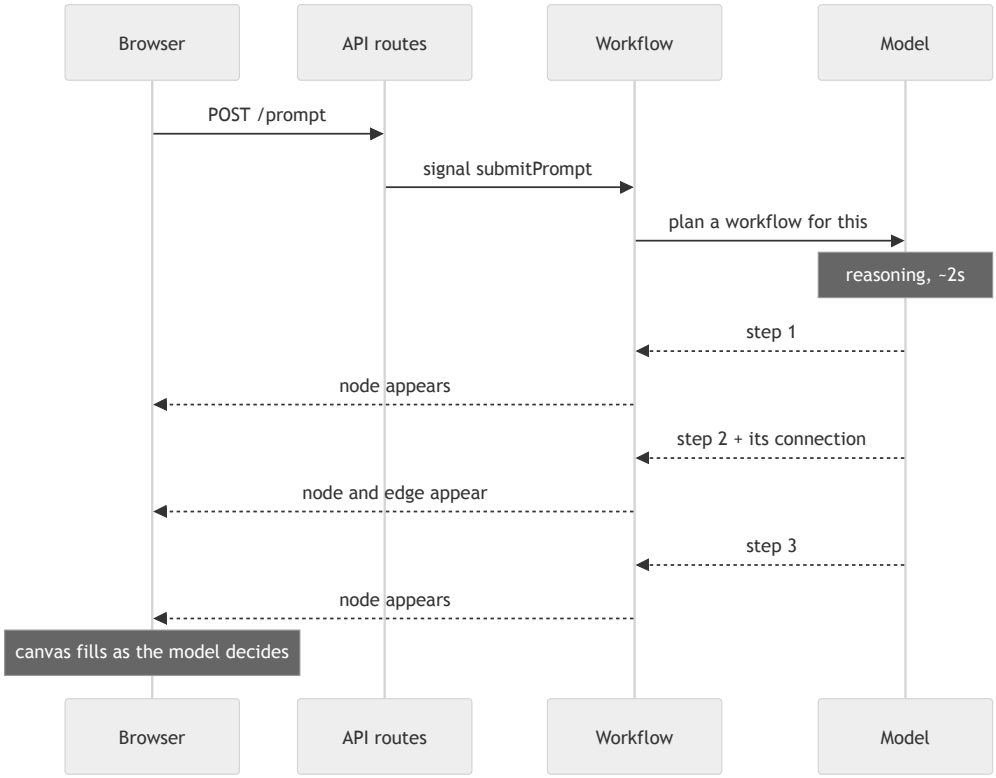
The whole thing at a glance



Steps 2 through 5 build the workflow. Steps 7 through 10 run a document through it. The two are separate, which is what makes a workflow worth keeping.

Stage 1: from a sentence to a workflow

You type *"summarise this contract and give me a Word version"* into the bar at the bottom of the canvas. That sentence does not touch a document. It builds the pipeline.



The model returns what the steps are and what feeds into what. Nothing else. Identifiers and screen positions are assigned by the workflow using a fixed layout function. If the model chose coordinates you would get overlapping boxes and the run would stop being reproducible.

Steps stream in one at a time, each with the connection into it, so the graph assembles itself instead of appearing all at once after a pause. On a real run the first step lands about 1.6 seconds in and the rest follow 120 to 370 milliseconds apart.

The model can only name tools that exist. Anything it invents is rejected before it reaches the canvas and you get a note in the chat instead of a broken step.

Stage 2: making it yours

The plan is a starting point. Every part of it is editable.

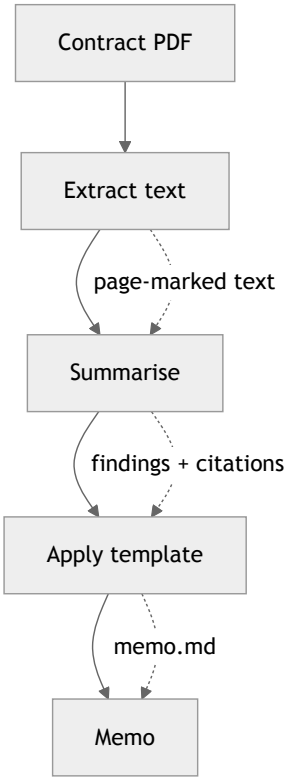
Action	Effect
Click a step	Panel opens with its settings
Edit a prompt	Sent to the workflow, confirmed, applied
Change a tool option	Form is generated from the tool, so options are always current
Draw a connection	Refused at the drag if the types cannot fit, with the reason shown
Delete a step	Confirmation names everything downstream that loses its input
Attach a document	Uploaded, hashed, referenced from then on

Connections are checked as you draw them. Each tool declares what it accepts and what it produces. Wiring a PDF into a step that only takes text is refused at the moment you try, instead of failing several minutes later mid run.

Edits are confirmed, not assumed. A change is only applied once the workflow accepts it. If the workflow refuses, you are told. An earlier version reported success either way, which meant the screen could show a graph the system had never agreed to.

Stage 3: the run

You attach a contract and press run.

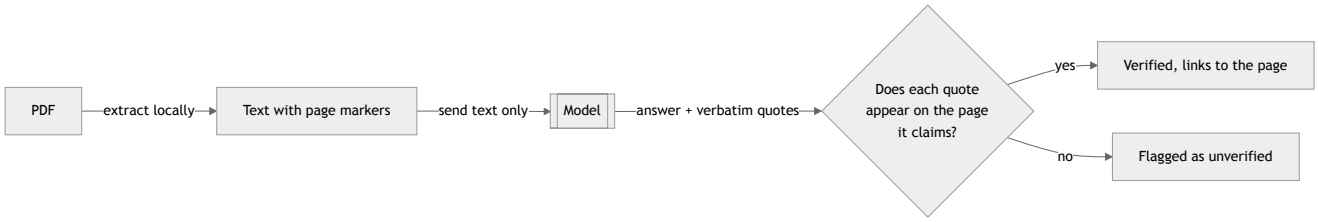


The workflow sorts the steps so nothing runs before its input exists, then runs independent branches at the same time. Each step retries on failure, times out, and can be cancelled, and its result is recorded before the next begins.

A failed step blocks everything downstream rather than letting it run against missing input, so the canvas shows exactly where it stopped.

File operations do not go through the model. Extracting, splitting, merging and converting are deterministic, so they run as ordinary code. The model is used for judgement only. That is faster, cheaper, reproducible, and fewer documents leave the machine.

Stage 4: why the output can be trusted



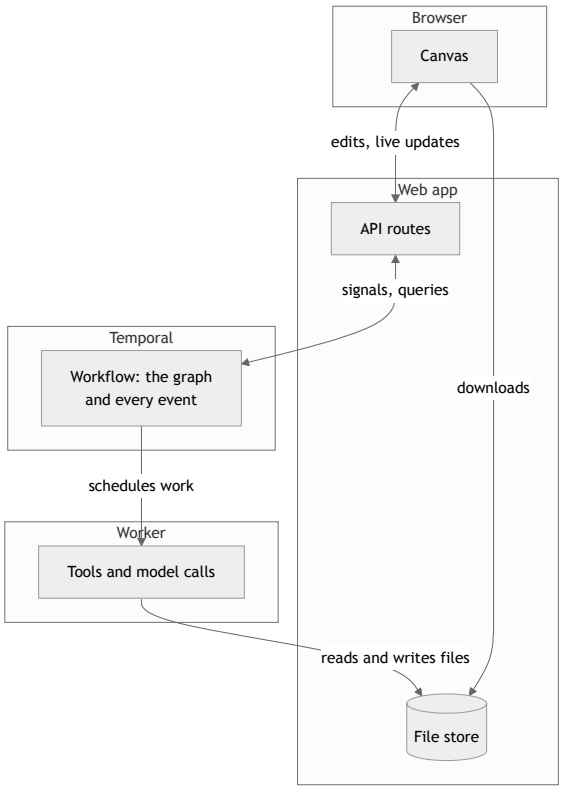
The model never receives the PDF. Text is extracted locally and tagged with page markers. The model must return verbatim quotes alongside its answer, each with a page number. Every quote is then matched back against the text of the page it claims.

A quote that matches becomes a link to that page of the source document. One that does not is shown with a warning.

A page number the model asserts is a claim. A page number we have matched is a fact. Treating those two things identically is how a confident, uncited paragraph ends up in front of a client.

An answer with no citations is labelled unsourced rather than shown plainly. The exported memo carries the same markers, so provenance survives being emailed on.

What each part does



Canvas. Draws the graph, handles dragging and connecting, renders results and citations. Decides nothing on its own.

API routes. Translate browser requests into workflow signals and queries, handle uploads and downloads, and stream events back to the canvas.

Workflow. Holds the authoritative graph and the full event history. Assigns identifiers and positions, validates every edit, and schedules the steps of a run in dependency order.

Worker. Runs the actual work: text extraction, splitting, merging, template filling, and model calls. Retries, times out and cancels under the workflow's control.

File store. Holds every document and artifact, addressed by content hash. Written by the worker, read by the browser.

Three properties follow from that split:

The workflow is the source of truth. The canvas holds a copy. Every change goes through the workflow and comes back as an event, so there is one path by which state changes and no way for the two to quietly disagree.

Documents never travel inside the workflow. They are written once, addressed by content hash, and everything afterwards passes the hash around. Large files stay out of the orchestration layer, and a reference recorded six months ago still names the exact bytes processed.

Every run is recorded. What was uploaded, which tool version handled it, which model was called, what came back, and whether each citation was verified.

What is enforced automatically

One command, `bun run e2e`, brings up the whole stack, runs 108 checks and tears it down. About 36 seconds.

- The built worker starts, not just compiles
- Both processes resolve the file store to the same place
- Planned steps only use real tools, and the graph has no cycles
- Steps arrive spread over time rather than in one batch
- Every edit operation genuinely changed the stored graph
- A deleted connection is not traversed during a run
- Every citation is re-checked with the harness's own matcher, rather than trusting the flag we set
- A citation the model was instructed to fabricate comes back unverified

The system that produces the verification is not allowed to be the system that confirms it.

Open questions

How long must a run record be kept? Everything needed for an audit is captured, but the current retention window is short and archiving is off. If the requirement is years, records need writing somewhere durable before real documents are involved.

Where does the model endpoint sit? Self hosted, a firm controlled proxy, and a third party API are very different disclosure profiles. The code does not care. The risk assessment does.

Where do documents rest once deployed? The file store is local disk. Split across two hosting providers there is no shared filesystem, so this needs object storage before anything ships.

Limitations

- The markdown output is a proof that the pipeline runs, not the intended product. Real output should be a Word file, a filled template or a populated form.
- The template step fills sections by matching headings, and the summarise step does not always produce headings that match, so parts of a generated memo can come back empty.
- Scanned documents with no text layer are refused rather than guessed at. No optical character recognition yet.
- Compression and Word conversion shell out to external programs that are not installed everywhere. They report clearly when missing rather than failing obscurely.
- No authentication. It is a proof of concept.